AJAX dan konsumsi API

Versi sederhana dengan contoh yang bisa dijalankan per submateri. Pelajari satu konsep, coba programnya, lalu ubah sedikit kodenya.

Untuk pengajar: panduan menjelaskan dengan bahasa sederhana, langkah demonstrasi, dan pertanyaan untuk praktikan tersedia pada halaman 13-15.

Setelah belajar, Anda dapat

- Meminta data tanpa memuat ulang seluruh halaman.

- Membaca data dengan XMLHttpRequest, Fetch, dan Axios.

- Mengirim data memakai POST serta menampilkan loading dan pesan gagal.

- Menerapkan alur yang sama pada komponen React.

Peta materi dan contoh program

Bagian	Inti pembelajaran	Contoh
7.1	Apa itu AJAX?	01
7.2	Cara kerja AJAX	02
7.3	Event handling dan XMLHttpRequest	03
7.4.1	GET dengan Fetch	04
7.4.2	GET dengan Axios	05
7.4.3	POST dengan Axios	06
7.5.1	Loading dan error handling	07
7.5.2	useState dan useEffect pada React	08
Pengayaan	API eksternal dan mock	09

Bekal awal

Anda sudah mengenal HTML, variabel, fungsi, event klik, dan array JavaScript. React baru dibutuhkan ketika mencapai contoh 08. Gunakan browser modern seperti Chrome, Edge, atau Firefox.

> Contoh 01-08 berjalan dengan data mock lokal tanpa internet. API publik pada contoh 09 bersifat pilihan. Semua pengguna dalam mock adalah data fiktif.

Materi disusun ulang dari Modul_07_AJAX_Konsumsi_API.pdf, bagian 7.1-7.5.2, halaman cetak 82-88. Paket ini menambahkan server latihan dan langkah percobaan yang dapat diulang.

PERSIAPAN

Jalankan dalam tiga langkah

1. Ekstrak paket

Ekstrak seluruh ZIP ke satu folder biasa. Jangan menjalankan file dari dalam arsip ZIP. Pertahankan susunan folder agar library dan data ditemukan.

2. Jalankan server

Instal Node.js LTS dari nodejs.org jika belum tersedia. Paket membutuhkan Node.js 18 atau lebih baru. Di Windows, klik dua kali JALANKAN.bat. Browser akan dibuka otomatis.

```
http://127.0.0.1:3007
```

Jika browser belum terbuka, ketik alamat tersebut di bilah alamat. Biarkan terminal server tetap terbuka. Untuk berhenti, kembali ke terminal dan tekan Ctrl+C.

3. Pilih contoh dari peta materi

Mulai dari contoh 01. Pada setiap contoh, jalankan tombol percobaan, baca hasilnya, buka bagian Lihat kode, lalu kerjakan latihan kecil. Edit file dengan VS Code atau editor teks, simpan, dan refresh browser.

File/folder	Fungsi
index.html	Halaman peta materi.
01-... sampai 09-...	Contoh per submateri: tampilan HTML dan kode program.
server.cjs	Server HTTP dan API mock.
vendor/	Axios, React, ReactDOM, Babel, dan font lokal.
assets/	CSS dan utilitas tampilan bersama.

Alternatif melalui terminal

Buka terminal di folder paket, lalu jalankan perintah berikut. Setelah itu buka alamat di atas secara manual.

```
node server.cjs
```

> Tidak perlu npm install, database, atau XAMPP. Membuka index.html langsung sebagai file:// hanya cocok untuk membaca peta; request dan API mock memerlukan server.cjs. Server statis biasa tidak menyediakan endpoint API paket ini.

Pahami AJAX dan alurnya

7.1. Apa itu AJAX?

AJAX adalah teknik bertukar data dengan server di belakang layar, lalu memperbarui sebagian halaman. Contoh: tombol Ambil pengguna menampilkan daftar baru sementara catatan yang sedang Anda ketik tetap ada.

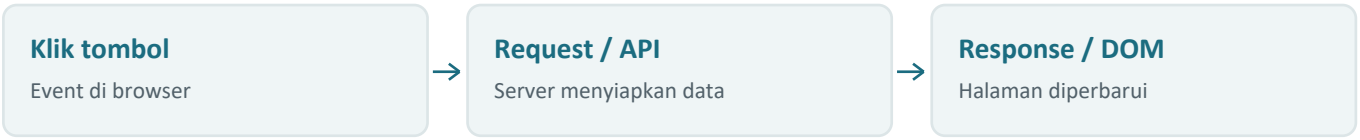
Kepanjangan AJAX adalah Asynchronous JavaScript and XML. Format datanya juga dapat berupa teks atau JSON. JSON adalah format teks untuk pertukaran data, misalnya objek nama dan email.

Buka contoh 01: 01-apa-itu-ajax/index.html. Kode utama: app.js di folder yang sama.

- Ketik catatan, lalu tekan Ambil pesan.

- Hasil: pesan dari server tampil, catatan dan waktu halaman dibuka tetap sama.

7.2. Cara kerja AJAX



- Event: pengguna menekan tombol.

- Request: JavaScript meminta data kepada server.

- Response: server mengirim status dan data.

- JavaScript membaca data, lalu memperbarui DOM (elemen halaman).

Asynchronous membuat browser tetap dapat menangani interaksi saat menunggu request. await menunggu hasil di dalam fungsi async yang memakainya; interaksi halaman lain masih dapat berjalan.

Buka contoh 02: 02-cara-kerja-ajax/index.html. Kode utama: app.js di folder yang sama.

- Tekan Mulai request, lalu tekan Tambah hitungan saat menunggu.

- Hasil: urutan request tercatat dan hitungan tetap bisa bertambah.

Kenali istilah yang akan dipakai

Istilah	Arti singkat
API	Cara program lain menyediakan layanan/data.
Endpoint	Alamat suatu layanan, misalnya /api/users.
GET / POST	GET membaca data; POST mengirim data untuk diproses.
DOM	Representasi elemen HTML yang dapat diubah JavaScript.

7.3 / CONTOH 03

Event dan XMLHttpRequest

Event handling berarti menjalankan fungsi ketika suatu kejadian terjadi. Di sini event click memulai XMLHttpRequest (XHR) untuk membaca ajax_info.txt.

Buka contoh 03: 03-event-xhr/index.html. Kode utama: app.js di folder yang sama.

Kode utama

```
const tombol = document.querySelector("#muat");
const hasil = document.querySelector("#hasil");

tombol.addEventListener("click", () => {
  tombol.disabled = true;
  hasil.textContent = "Memuat teks...";
  const xhr = new XMLHttpRequest();
  xhr.onreadystatechange = () => {
    if (xhr.readyState !== 4) return;
    if (xhr.status >= 200 && xhr.status < 300) {
      hasil.textContent = xhr.responseText;
    } else if (xhr.status !== 0) {
      hasil.textContent = `Gagal: HTTP ${xhr.status}`;
    }
  };
  xhr.onerror = () => { hasil.textContent = "Koneksi gagal."; };
  xhr.ontimeout = () => { hasil.textContent = "Waktu tunggu habis."; };
  xhr.onloadend = () => { tombol.disabled = false; };
  // open menyiapkan request; send yang mengirimkannya.
  xhr.open("GET", "ajax_info.txt", true);
  xhr.timeout = 5000;
  xhr.send();
});
```

Bagian kode	Makna
addEventListener	Hubungkan klik tombol dengan fungsi.
open(..., true)	Siapkan metode, URL, dan mode asynchronous.
send()	Kirim request yang telah disiapkan.
readyState === 4	Proses request selesai; belum berarti berhasil.
status 200-299	Response berhasil. Status 404 berarti file tidak ditemukan.
responseText	Isi response dalam bentuk teks.

Coba dan amati

Tekan Muat teks. Isi file muncul sebagai teks. Ubah isi ajax_info.txt, simpan, lalu klik lagi. Ubah URL menjadi tidak-ada.txt untuk mencoba pesan HTTP 404, kemudian kembalikan.

textContent menampilkan data sebagai teks. Hindari memasukkan data dari API yang belum dipercaya langsung ke innerHTML.

7.4.1 / CONTOH 04

GET dengan Fetch

Fetch sudah tersedia di browser modern. Contoh ini mengambil array pengguna dari API mock lokal, membaca JSON, dan membuat elemen daftar.

Buka contoh 04: 04-fetch-get/index.html. Kode utama: app.js di folder yang sama.

Kode utama

```
const tombol = document.querySelector("#ambil");
const status = document.querySelector("#status");
const daftar = document.querySelector("#daftar");

tombol.addEventListener("click", async () => {
  tombol.disabled = true;
  status.textContent = "Memuat pengguna...";
  daftar.replaceChildren();
  try {
    const response = await fetch("/api/users?delay=700");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const users = await response.json();
    for (const user of users) {
      const item = document.createElement("li");
      item.textContent = `${user.name} - ${user.email}`;
      daftar.append(item);
    }
    status.textContent = users.length ? `${users.length} pengguna dimuat.` : "Data kosong.";
  } catch (error) {
    status.textContent = `Gagal memuat: ${error.message}`;
  } finally {
    tombol.disabled = false;
  }
});
```

Baca tiga baris terpenting

Kode	Makna
await fetch(url)	Tunggu Response dari server.
if (!response.ok) ...	Periksa status HTTP. Fetch tidak otomatis menolak HTTP 404/500.
await response.json()	Baca isi JSON menjadi array/objek JavaScript.

Coba dan amati

Tekan Ambil pengguna. Tiga nama dan email muncul. Klik Lihat JSON mentah untuk melihat data sebelum ditampilkan. Ubah item.textContent menjadi user.name, simpan, lalu refresh untuk menampilkan nama saja.

> Contoh bentuk data: [{ "id": 1, "name": "Budi Santoso", "email": "budi@example.com" }]. API mock mengirim tiga objek seperti ini.

7.4.2 / CONTOH 05

GET dengan Axios

Axios adalah library untuk request HTTP. Pada response JSON, data langsung tersedia di response.data. Paket memuat vendor/axios.min.js dari penyimpanan lokal.

Buka contoh 05: 05-axios-get/index.html. Kode utama: app.js di folder yang sama.

Kode utama

```
const tombol = document.querySelector("#ambil");
const status = document.querySelector("#status");
const daftar = document.querySelector("#daftar");

tombol.addEventListener("click", async () => {
  tombol.disabled = true;
  status.textContent = "Memuat dengan Axios...";
  daftar.replaceChildren();
  try {
    const response = await axios.get("/api/users", { timeout: 5000 });
    const users = response.data;
    for (const user of users) {
      const item = document.createElement("li");
      item.textContent = `${user.name} - ${user.email}`;
      daftar.append(item);
    }
    status.textContent = users.length ? `${users.length} pengguna dimuat.` : "Data kosong.";
  } catch (error) {
    status.textContent = `Gagal: ${error.response?.status || error.message}`;
  } finally {
    tombol.disabled = false;
  }
});
```

Fetch dan Axios: pilih yang paling jelas bagi Anda

Aspek	Fetch	Axios
Ketersediaan	Bawaan browser	Library tambahan
Membaca JSON	await response.json()	response.data
HTTP 4xx/5xx	Periksa response.ok	Ditolak secara bawaan
Timeout	AbortController + timer	Opsi timeout
Pembatalan	AbortController	Mendukung AbortController

Coba dan amati

Tekan Ambil dengan Axios. Hasilnya sama dengan Fetch. Untuk mencoba error, ubah URL menjadi /api/users?mode=error, simpan, refresh, lalu klik tombol. Setelah itu kembalikan URL semula.

Pada proyek dengan npm, Axios biasa dipasang memakai npm install axios lalu diimpor. Langkah itu tidak diperlukan dalam paket latihan ini karena file library sudah disertakan.

7.4.3 / CONTOH 06

Kirim data memakai POST

POST mengirim data ke server. Form menyediakan nama dan email; event.preventDefault() mencegah form memuat ulang halaman. Axios mengirim objek biasa ini sebagai JSON.

Buka contoh 06: 06-axios-post/index.html. Kode utama: app.js di folder yang sama.

Kode inti di dalam handler submit

```
form.addEventListener("submit", async (event) => {
  event.preventDefault();
  const userBaru = {
    name: document.querySelector("#nama").value.trim(),
    email: document.querySelector("#email").value.trim()
  };
  try {
    const response = await axios.post(
      "/api/users", userBaru, { timeout: 5000 }
    );
    hasil.textContent = JSON.stringify(response.data, null, 2);
  } catch (error) {
    status.textContent = "Pengiriman gagal.";
  }
});
```

Cuplikan menunjukkan alur inti. File app.js lengkap menambahkan validasi nama, status loading, status HTTP, dan pengaktifan kembali tombol di finally. Form HTML tersedia di index.html.

Coba dan amati

- Isi Budi Praktikan dan budi@example.com, lalu tekan Kirim data.

- Hasil: HTTP 201 dan response JSON berisi data yang dikirim.

- Buka DevTools > Network, pilih request users, dan lihat Payload.

```
{
  "id": 101,
  "name": "Budi Praktikan",
  "email": "budi@example.com",
  "simulated": true
}
```

> POST ini hanya simulasi. Server tidak menyimpan data, dan GET /api/users tetap mengembalikan tiga pengguna awal. Status 201 digunakan untuk meniru response pembuatan data.

Latihan kecil

Kirim nama berbeda. Cocokkan isi input, payload request, dan response. Nama yang hanya berisi spasi ditolak; alamat email harus memiliki format email.

7.5.1 / CONTOH 07

Loading dan error handling

Tampilkan keadaan proses agar pengguna tahu apa yang terjadi: sedang memuat, berhasil, kosong, atau gagal. Timeout berarti request melewati batas waktu yang ditentukan.

Buka contoh 07: 07-loading-error/index.html. Kode utama: app.js di folder yang sama.

Pola inti yang dipakai

```
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), batas);
tombol.disabled = true;
status.textContent = "Memuat data...";
try {
  const response = await fetch(url, { signal: controller.signal });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const users = await response.json();
  // Tampilkan users ke daftar (lihat app.js lengkap).
  status.textContent = users.length ? "Berhasil." : "Data kosong.";
} catch (error) {
  status.textContent = error.name === "AbortError"
    ? "Waktu tunggu habis. Silakan coba lagi."
    : `Gagal memuat: ${error.message}`;
} finally {
  clearTimeout(timer);
  tombol.disabled = false;
}
```

try menjalankan request. catch menangani kegagalan. finally membersihkan timer dan mengaktifkan tombol, baik request berhasil maupun gagal.

Empat percobaan yang dapat diulang

Pilihan	Yang dilakukan server/browser	Hasil
Berhasil	Response 200 setelah sekitar 900 ms	Daftar pengguna tampil.
Error HTTP 503	Server sengaja mengirim 503	Pesan gagal tampil.
Data kosong	Response 200 berisi []	Pesan data kosong tampil.
Timeout	Batas 700 ms, response setelah 2500 ms	Request dibatalkan.

Latihan kecil

Di app.js, ubah batas timeout dari 700 menjadi 3000. Jalankan skenario Timeout lagi. Response akan tiba sebelum batas baru, sehingga hasilnya berhasil.

> Data kosong tetap merupakan response berhasil. Error jaringan, error HTTP, dan timeout punya penyebab berbeda; tampilkan pesan yang membantu pengguna mencoba lagi.

7.5.2 / CONTOH 08 / BAGIAN 1

Ambil data pada React

useState menyimpan data yang memengaruhi tampilan. useEffect menjalankan request setelah render dan saat dependensinya berubah. React merender ulang tampilan ketika state diperbarui.

Buka contoh 08: 08-react-state/index.html. Kode utama: app.jsx di folder yang sama.

State dan effect di app.jsx

```
const { useState, useEffect } = React;

function DaftarUser() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [mode, setMode] = useState("success");
  const [percobaan, setPercobaan] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    let aktif = true;
    async function ambilData() {
      setLoading(true);
      setError("");
      setUsers([]);
      try {
        const response = await axios.get(`/api/users?mode=${mode}&delay=800`, {
          timeout: 5000, signal: controller.signal
        });
        if (aktif) setUsers(response.data);
      } catch (err) {
        if (aktif && !axios.isCancel(err)) setError("Gagal memuat. Silakan coba lagi.");
      } finally {
        if (aktif) setLoading(false);
      }
    }
    ambilData();
    return () => {
      aktif = false;
      controller.abort();
    };
  }, [mode, percobaan]);
```

State	Tugas
users	Array data yang akan ditampilkan.
loading	true selama request masih berjalan.
error	Pesan gagal; string kosong jika tidak ada error.
mode / percobaan	Pemicu effect ketika pilihan berubah atau tombol dimuat ulang.

Lanjutan JSX ada di halaman berikutnya. Library React, ReactDOM, Axios, dan Babel lokal sudah dimuat oleh index.html; Anda tidak perlu memasang paket npm.

Render hasil dan rapikan request

Lanjutan komponen: pilih tampilan sesuai state

```
return (
    <section aria-busy={loading}>
      <label htmlFor="mode-react">Skenario response</label>
      <select id="mode-react" value={mode} onChange={e => setMode(e.target.value)}>
        <option value="success">Berhasil</option>
        <option value="error">Error HTTP 503</option>
        <option value="empty">Data kosong</option>
      </select>
      <button disabled={loading} onClick={() => setPercobaan(n => n + 1)}>
        Muat ulang data
      </button>
      <p role="status" className="result" data-state={error ? "error" : "success"}>
        {loading ? "Memuat data..." : error || (users.length ? "Data siap." : "Data kosong.")}
      </p>
      {!loading && !error && <ul className="users">
        {users.map(user => <li key={user.id}>{user.name} - {user.email}</li>)}
      </ul>}
    </section>
  );
}

ReactDOM.createRoot(document.querySelector("#root")).render(<DaftarUser />);
```

Cara membaca alurnya

- Effect meminta data dan mengaktifkan loading.

- setUsers menyimpan data; setError menyimpan pesan gagal. finally menutup loading.

- JSX menampilkan hasil sesuai state. key memakai id untuk mengenali tiap item.

Mengapa ada cleanup?

Saat mode berubah atau komponen dilepas, cleanup menonaktifkan effect lama dan membatalkan request-nya. Response lama tidak akan menimpa hasil request baru.

> Effect dengan [] berjalan setelah mount, termasuk saat dipasang ulang. StrictMode saat pengembangan dapat menjalankan setup/cleanup tambahan. Contoh ini memakai [mode, percobaan] agar perubahan pilihan dan klik muat ulang juga memicu effect.

Coba dan amati

Ganti pilihan menjadi Error HTTP 503, Data kosong, lalu Berhasil. Tekan Muat ulang data. Coba juga mengganti mode saat loading.

API eksternal dan mock

Mock adalah layanan tiruan untuk latihan. API eksternal berjalan di server pihak lain. Keduanya memakai pola request yang sama; perbedaannya ada pada URL, ketersediaan jaringan, dan kebijakan server.

Buka contoh 09: 09-api-eksternal/index.html. Kode utama: app.js di folder yang sama.

Ganti sumber, pertahankan alur Fetch

```
const url = lokal
  ? "/api/users"
  : "https://jsonplaceholder.typicode.com/users";
const response = await fetch(url, { signal: controller.signal });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const users = await response.json();
// Program contoh menampilkan maksimal tiga pengguna pertama.
for (const user of users.slice(0, 3)) {
  const item = document.createElement("li");
  item.textContent = `${user.name} - ${user.email}`;
  daftar.append(item);
}
```

Cuplikan ini adalah bagian inti. File app.js lengkap mencakup tombol, pilihan sumber, batas waktu 8 detik, dan pesan jika request gagal.

Endpoint mock yang tersedia

Request	Response
GET /api/info	200, objek berisi message.
GET /api/users	200, array tiga pengguna.
GET /api/users?mode=empty	200, array kosong.
GET /api/users?mode=error	503, simulasi gagal.
POST /api/users	201, data simulasi tanpa penyimpanan.

Parameter delay mengatur jeda dalam milidetik, dibatasi 0-5000. Misalnya /api/users?delay=1500 akan merespons setelah sekitar 1,5 detik.

Coba dan amati

- Pilih Mock lokal dan jalankan; internet tidak diperlukan.

- Pilih JSONPlaceholder saat internet tersedia. Lihat tiga pengguna pertamanya.

- Jika layanan eksternal gagal, pilih mock secara manual untuk melanjutkan.

> CORS adalah izin dari server untuk request lintas origin. Jika API menolak CORS, JavaScript browser tidak dapat memberi izin itu sendiri. mode: "no-cors" menghasilkan response yang tidak dapat dibaca sebagai JSON.

Latihan dan bantuan cepat

Tugas mandiri

- Fetch: tampilkan nama dan email, lalu tambahkan jumlah pengguna di bawah daftar.

- POST: kirim data contoh lain dan buktikan bahwa response cocok dengan payload.

- Error: coba semua skenario pada contoh 07. Catat pesan dan status HTTP-nya.

- React: ubah teks loading, lalu coba Berhasil, Error, dan Data kosong.

Cek pemahaman dan jawaban singkat

Pertanyaan	Jawaban
Apa yang diubah setelah AJAX selesai?	Elemen DOM yang dipilih oleh program.
Apakah readyState 4 menjamin berhasil?	Tidak. Tetap periksa status HTTP.
Mengapa Fetch memeriksa response.ok?	HTTP 4xx/5xx tidak otomatis menolak Promise Fetch.
Mengapa ada finally?	Untuk merapikan keadaan setelah berhasil atau gagal.
Mengapa POST tidak muncul pada GET?	Mock ini tidak menyimpan data.

Jika program belum berjalan

Gejala	Langkah perbaikan
node tidak dikenali	Instal Node.js LTS, lalu buka ulang terminal.
Situs tidak dapat dijangkau	Jalankan server.cjs dan periksa alamat/port.
Port 3007 terpakai	Tutup server lama; pilihan port lain ada di README.
Membuka lewat file://	Gunakan http://127.0.0.1:3007.
JSON tidak terbaca	Periksa URL dan isi response di Network.
Error API eksternal	Cek koneksi/layanan, lalu coba mock lokal.

Bacaan lanjut

MDN: developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
Axios: axios-http.com/docs/intro
React: react.dev/reference/react/useEffect
API latihan: jsonplaceholder.typicode.com/guide/

Sumber materi: Modul_07_AJAX_Konsumsi_API.pdf, halaman cetak 82-88. Penjelasan diperbarui untuk membedakan open dan send pada XHR, pemeriksaan status Fetch, serta cleanup effect React. Versi library dan lisensinya tercantum di README dan folder vendor.

Jelaskan dari hal yang terlihat

Mulai dari hasil di browser, kemudian hubungkan dengan istilah dan kode. Jangan menampilkan seluruh kode sekaligus. Pada setiap contoh, gunakan urutan: jelaskan tujuan, jalankan program, amati hasil, lalu baca kode inti.

Contoh 01 / Apa itu AJAX?

> Ucapkan: "Biasanya, ketika halaman di-refresh, seluruh halaman dimuat ulang. Dengan AJAX, kita bisa mengambil data baru dan mengubah bagian tertentu saja."

Demonstrasi: minta praktikan mengetik catatan, lalu menekan Ambil pesan. Arahkan perhatian pada pesan yang berubah dan catatan yang tetap ada.

Lanjutkan: "Pesannya berubah, tetapi catatan kalian tetap ada. Inilah yang kita pelajari: mengambil data tanpa me-refresh seluruh halaman."

Contoh 02 / Cara kerja AJAX

> Ucapkan: "Alurnya sederhana: tombol diklik, browser meminta data, server menjawab, lalu JavaScript menampilkan jawabannya."

Demonstrasi: tekan Mulai request. Selama menunggu, minta praktikan menekan Tambah hitungan. Tunjukkan bahwa hitungan masih bisa bertambah.

Ucapkan: "Tombol lain masih bisa digunakan sambil menunggu data. Inilah maksud asynchronous pada percobaan ini."

Istilah setelah demonstrasi	Bahasa sederhana
Request	Permintaan dari browser.
Response	Jawaban dari server.
DOM	Elemen halaman yang dapat diubah JavaScript.

Contoh 03 / Event dan XMLHttpRequest

> Ucapkan: "Event itu kejadian, misalnya tombol diklik. Kita mengatur agar kejadian klik menjalankan fungsi untuk meminta file."

Demonstrasi: tekan Muat teks, lalu tunjukkan bahwa isinya berasal dari ajax_info.txt. Jelaskan open() untuk menyiapkan permintaan, send() untuk mengirim, dan responseText untuk membaca teks jawaban.

Ubah alamat ke file yang tidak ada untuk memperlihatkan HTTP 404. Tekankan: "Proses selesai belum tentu berhasil. Karena itu, kita juga memeriksa status HTTP." Setelah percobaan, kembalikan alamat file semula.

Membaca dan mengirim data

Contoh 04 / Fetch

> Ucapkan: "Sekarang kita meminta daftar pengguna. Data dari server berbentuk JSON, yaitu format teks terstruktur untuk bertukar data."

Demonstrasi: tekan Ambil pengguna. Setelah daftar muncul, buka JSON mentahnya. Baru kemudian tunjukkan tiga baris kode berikut.

```
const response = await fetch("/api/users");
if (!response.ok) throw new Error("Request gagal");
const users = await response.json();
```

Baris	Cara menjelaskannya
1	Meminta data dan menunggu Response dari server.
2	Memeriksa apakah status HTTP berhasil.
3	Membaca JSON menjadi data yang bisa diolah JavaScript.

Tekankan bahwa HTTP 404/500 pada Fetch perlu diperiksa lewat response.ok. Setelah data terbaca, program membuat elemen daftar untuk menampilkannya.

Contoh 05 / Axios

> Ucapkan: "Axios membantu kita melakukan request HTTP. Tujuannya sama seperti Fetch. Axios menyediakan beberapa kemudahan saat membaca data dan menangani kegagalan."

Demonstrasi: tekan Ambil dengan Axios, lalu bandingkan hasil dan kode dengan contoh Fetch. Cukup tekankan dua hal terlebih dahulu:

- Data JSON tersedia melalui response.data.

- Secara bawaan, status HTTP di luar 200-299 ditolak dan ditangani di catch.

Praktikan tidak perlu menghafal semua perbedaan. Minta mereka menemukan bagian kode yang meminta data dan bagian yang membaca jawabannya.

Contoh 06 / POST

> Ucapkan: "Tadi kita membaca data dengan GET. Sekarang kita mengirim data dengan POST. Isi formulir menjadi data yang dikirim; server memprosesnya, lalu memberikan jawaban."

Demonstrasi: isi nama dan email, tekan Kirim data, lalu cocokkan isi formulir dengan response. Jika praktikan sudah siap, buka DevTools > Network dan perlihatkan Payload.

Tegaskan: "Pada latihan ini, pengiriman hanya simulasi. Datanya dikembalikan sebagai jawaban, tetapi belum disimpan ke database. Karena itu, daftar GET tetap berisi tiga pengguna awal."

Jelaskan kondisi dan perubahan

Contoh 07 / Loading dan error

> Ucapkan: "Request membutuhkan waktu dan bisa gagal. Pengguna perlu diberi tahu apa yang sedang terjadi."

Demonstrasi: jalankan semua pilihan skenario. Amati teks Memuat sebelum hasil akhir muncul.

Kondisi	Penjelasan untuk praktikan
Loading	Sedang menunggu jawaban.
Berhasil	Data berhasil diterima.
Kosong	Request berhasil, tetapi tidak ada item.
Error	Permintaan gagal, misalnya HTTP 503.
Timeout	Batas waktu menunggu sudah habis.

Ucapkan: "try untuk mencoba pekerjaan, catch untuk menangani kegagalan, dan finally untuk merapikan keadaan setelah selesai." Tunjukkan bahwa tombol aktif kembali setelah berhasil maupun gagal.

Contoh 08 / React

> Ucapkan: "Alur request-nya tetap sama. Di React, kita menyimpan kondisi tampilan dalam state. Ketika state diperbarui, React memperbarui tampilannya."

Kenalkan users sebagai daftar data, loading sebagai penanda menunggu, dan error sebagai pesan gagal. Jelaskan bahwa useEffect menjalankan pengambilan data setelah render, termasuk saat pilihan skenario berubah pada contoh ini.

Demonstrasi: coba Berhasil, Error HTTP 503, Data kosong, dan Muat ulang data. Bahas cleanup setelah alur dasar dipahami: request lama dibatalkan agar hasil lama tidak menimpa hasil baru.

Contoh 09 / API lokal dan eksternal

> Ucapkan: "API lokal berjalan di komputer kita. API eksternal berjalan di server pihak lain. Cara memintanya mirip, tetapi alamat dan kebutuhan koneksinya berbeda."

Demonstrasi: jalankan Mock lokal dahulu. Ketika internet tersedia, pilih JSONPlaceholder. Jika layanan eksternal gagal, pilih mock secara manual untuk melanjutkan latihan.

Pertanyaan setelah setiap contoh

- Apa pemicunya?

- Data datang dari mana?

- Bagian halaman mana yang berubah?

Minta praktikan mengubah satu hal kecil dan menjelaskan hasilnya sebelum melanjutkan ke contoh berikutnya. Misalnya mengganti teks tombol, hanya menampilkan nama, atau mencoba URL yang salah.